FR5 Board Stacking: Robot Handoff

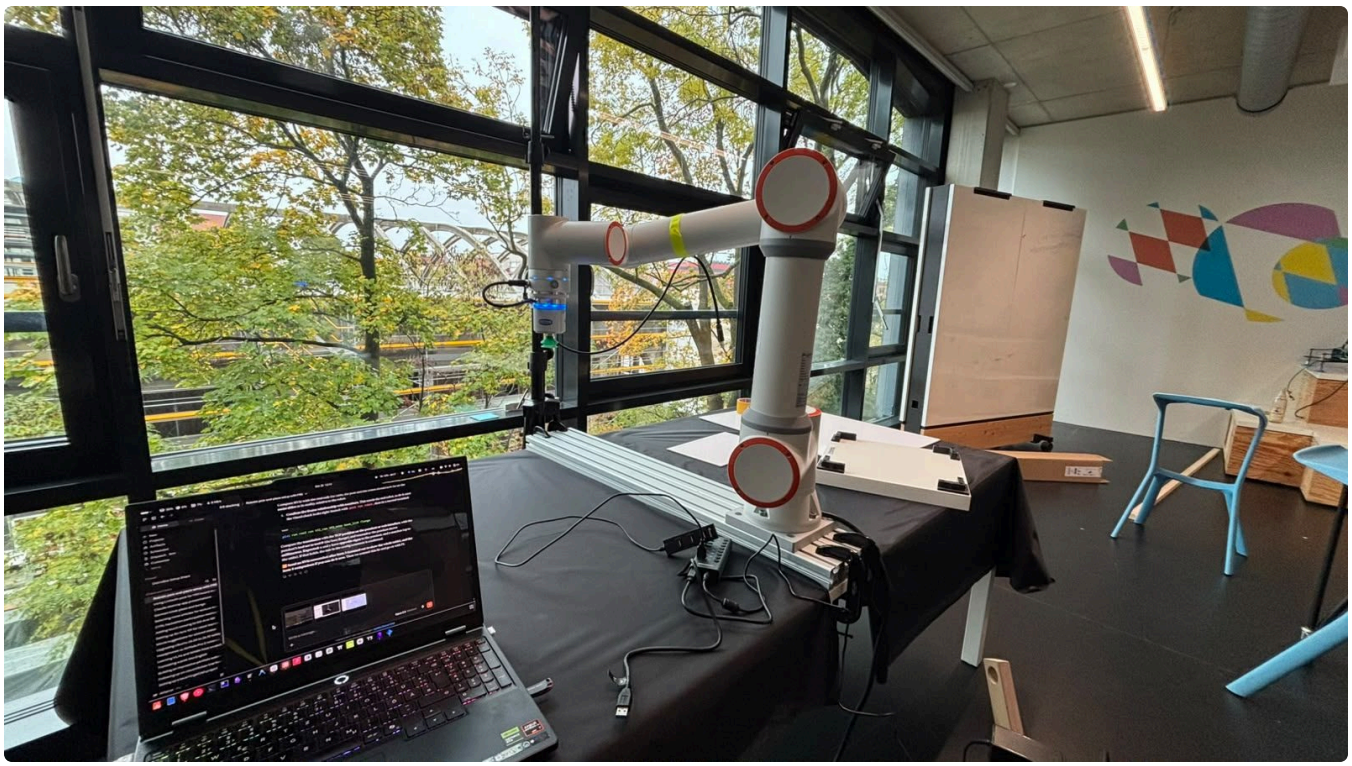
Oct 25, 2026 · @Atharva B

The FR5 now moves under MoveIt on the real robot, the vacuum switches from ROS, and the workcell (table, post, zones, cameras) is modelled. This doc is everything needed to take over the robot and task side while vision is built in parallel.

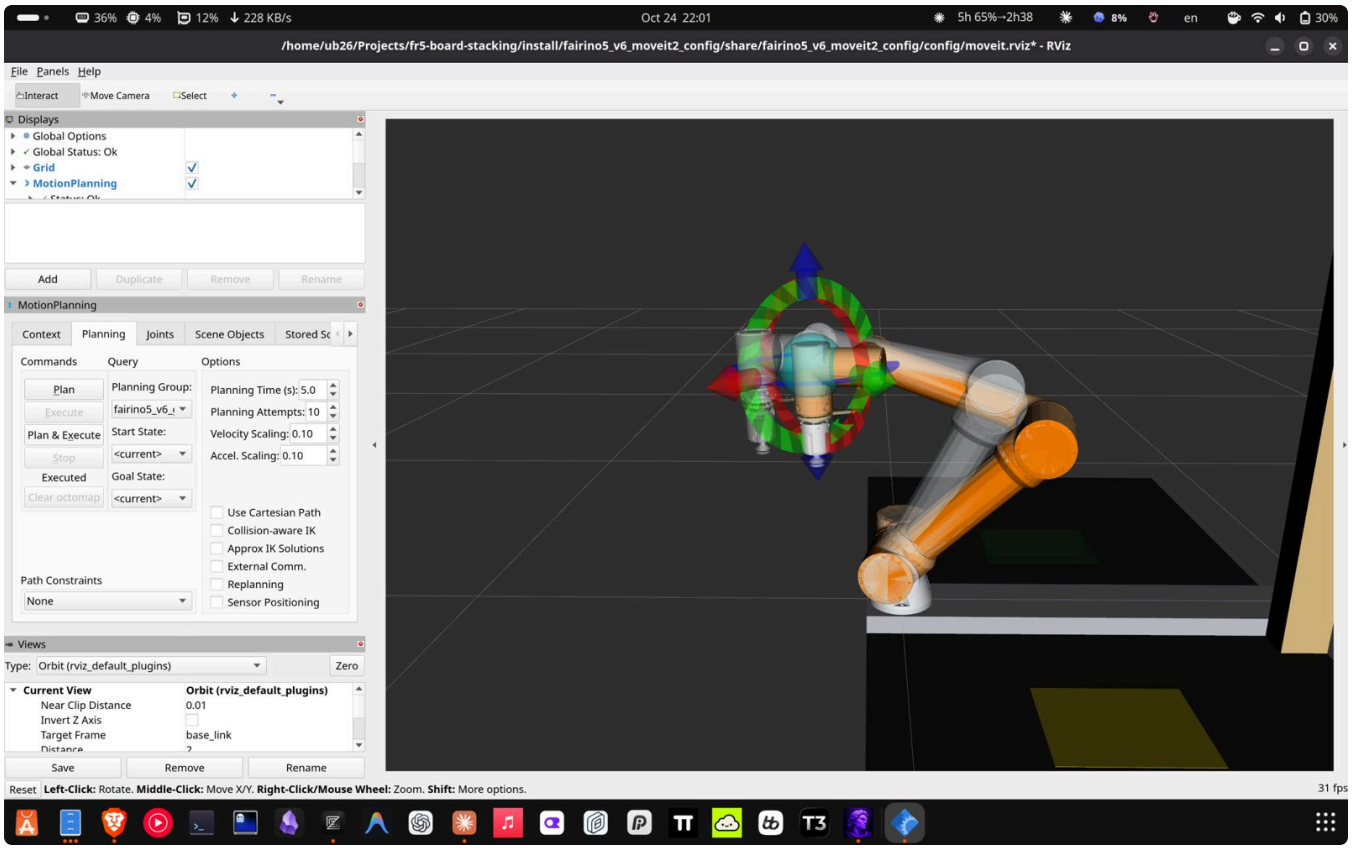
Status at a glance

Motion and vacuum both work on the real robot; the missing piece is the task code that chains them into a pick-and-place.

Capability	Sim	Real robot	Notes
MoveIt planning and execution	✅	✅	Tested at 10% scaling; joints match the web UI exactly
Robot model: FR5 + D405 mount + Schmalz gripper + `tcp`	✅	✅	`tcp` = relaxed cup tip, 112.9 mm from the flange
Workcell: table, profile, camera post, zones	✅	✅	Collision-aware; orientation fixed to match reality
Vacuum on/off from ROS (`/gripper/set_vacuum`)	✅ fake	✅	Via the controller's XML-RPC, coexists with motion
Part-present / vacuum feedback	❌	❌	Not available yet (see TODOs)
Pick-and-place sequence (P1)	Code drafted	❌	Your next task; code in the appendix
Vision	—	—	Being built in parallel (`feat/perception-*`)



The cell today: FR5 on the aluminium profile, wooden camera post at the far end.



RViz during the first real-robot moves: the orange goal state and the executed pose.

Hardware

What's on the robot and what we pick, so the model in RViz makes sense.

Item	Details
Robot	Fairino FR5 (5 kg payload, 922 mm reach), controller V3.9.2, robot V6.0
Gripper	Schmalz electric cobot vacuum gripper, single bellows cup, switched by tool DO0
Wrist camera	Intel RealSense D405 on a 3D-printed plate between flange and gripper (adds 9.5 mm)
Overhead camera	Logitech webcam on the wooden post, 1.61 m above the table, looking down
Boards	27 × 20 cm, 6 mm thick, white on both sides


FAIRINO
Name : Collaborative robot
Model : FR5
Payload : 5kg
Reach : 922mm
Weight : 22kg
Input power : 48VDC
IP Classification : IP65
Output short circuit : 42Amax
SN: SN.126023-0025165-01
RoHS NSF
Manufacturer:
FAIR Innovation (Suzhou) Robot System Co, Ltd.
Address:
No.36 Zijing Road,Wuzhong District,Suzhou,Jiangsu,
PR.China
No.5888Zunxian Road,New District,Zibo,Shandong,
PR.China
Email: sales@frtech.fr
Web: www.fairino.com

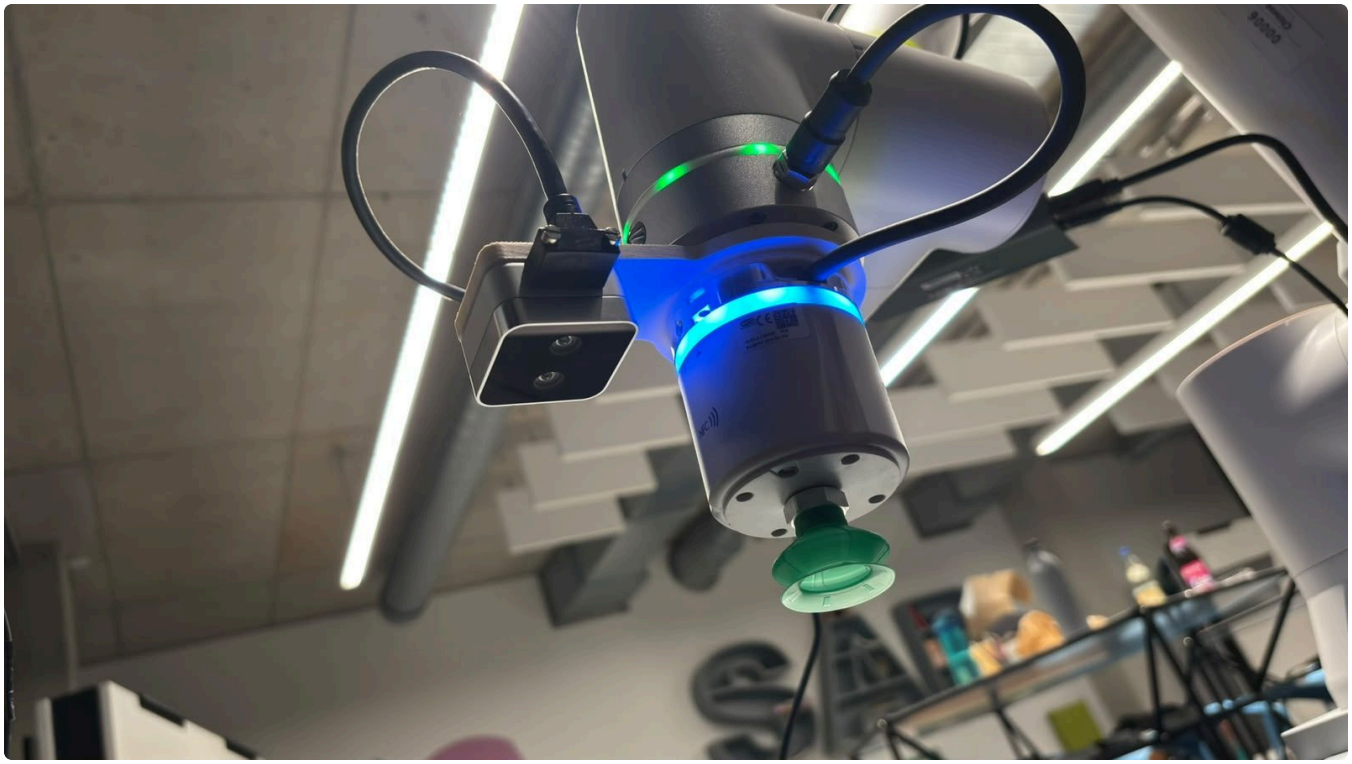


Robot label.

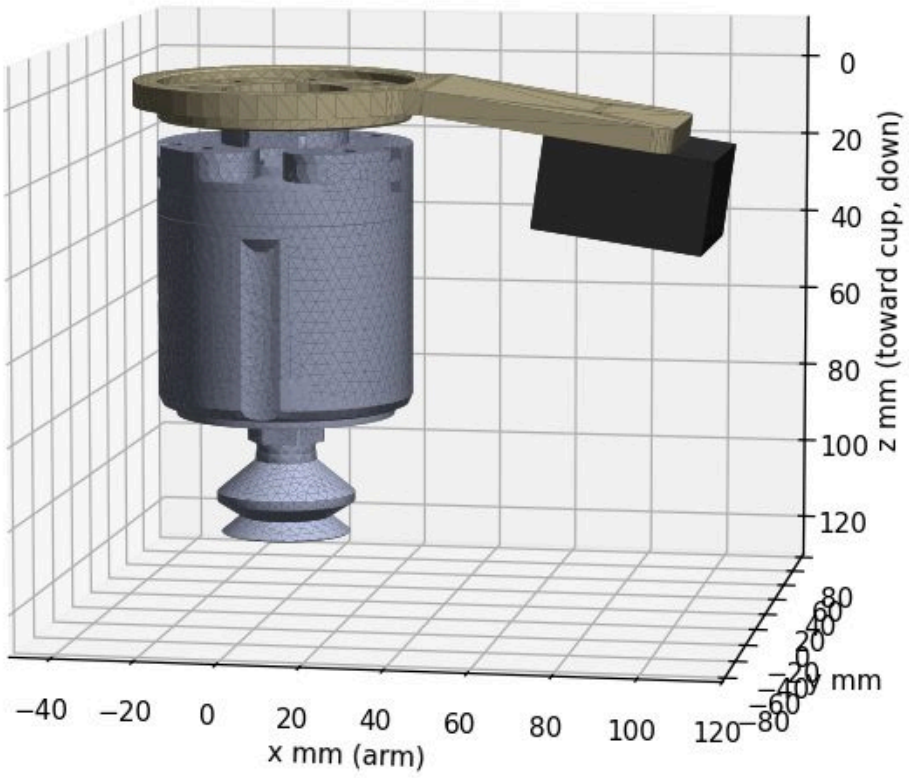




Gripper and D405 mount, side view: the printed plate sits between the flange and the gripper.



From below: the D405 looks down beside the cup.

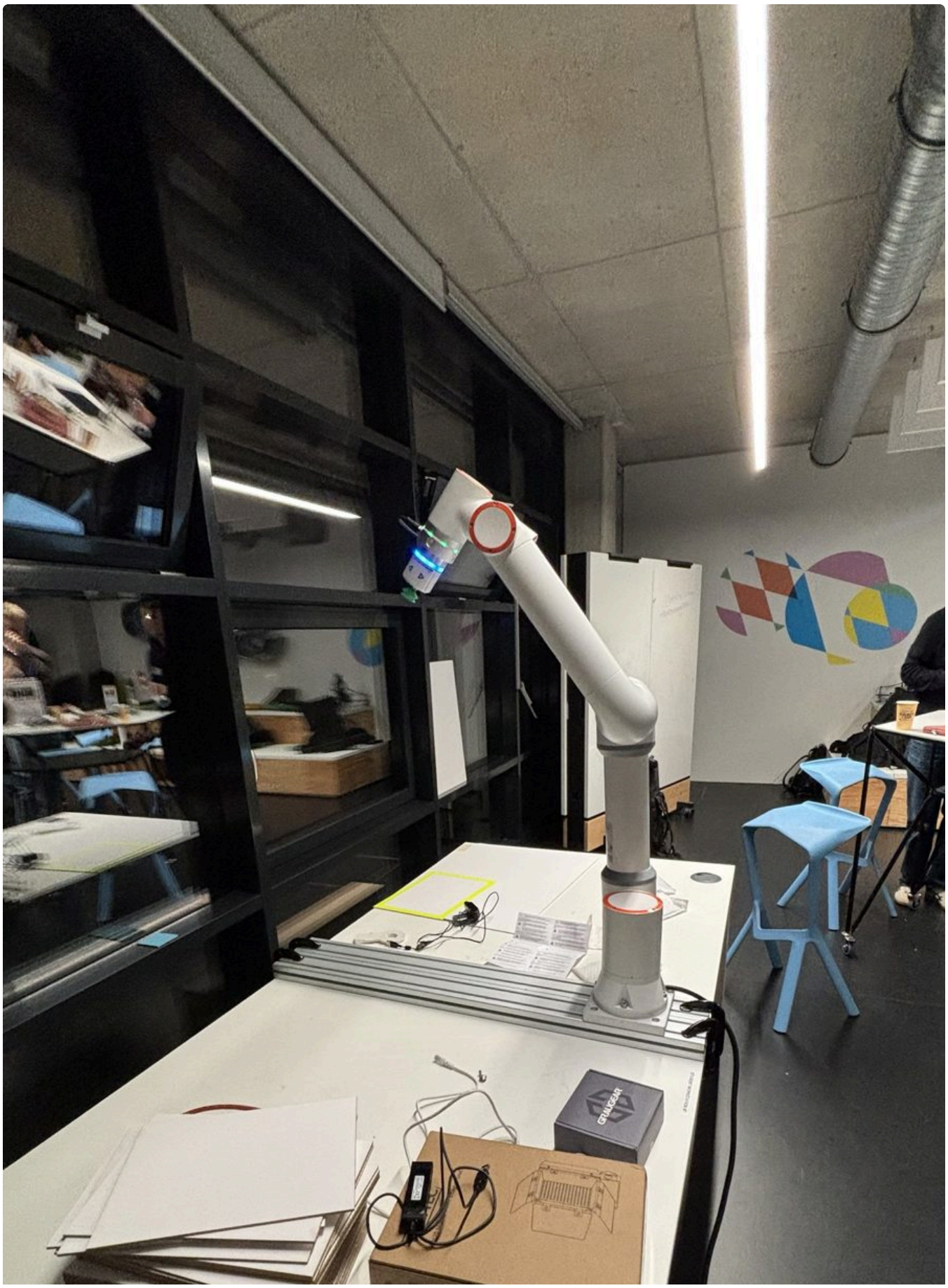


Our CAD check of the stack (flange at z = 0, cup tip at 112.9 mm), matching the photos.





The boards to pick and stack.


GRIPPER



The cell on day 1, before the overhead camera post.

Quick start

Everything runs through pixi; no system ROS install is needed.

Setup (once)

```
git clone --recurse-submodules https://github.com/AtharvaBhorpe/fr5-board-stacking.git
cd fr5-board-stacking
pixi install
pixi run build
```

Existing clone: `git switch main && git pull && git submodule update --init && pixi run build`. Rebuild after any pull that adds files under `src/`.

Simulation (safe anywhere)

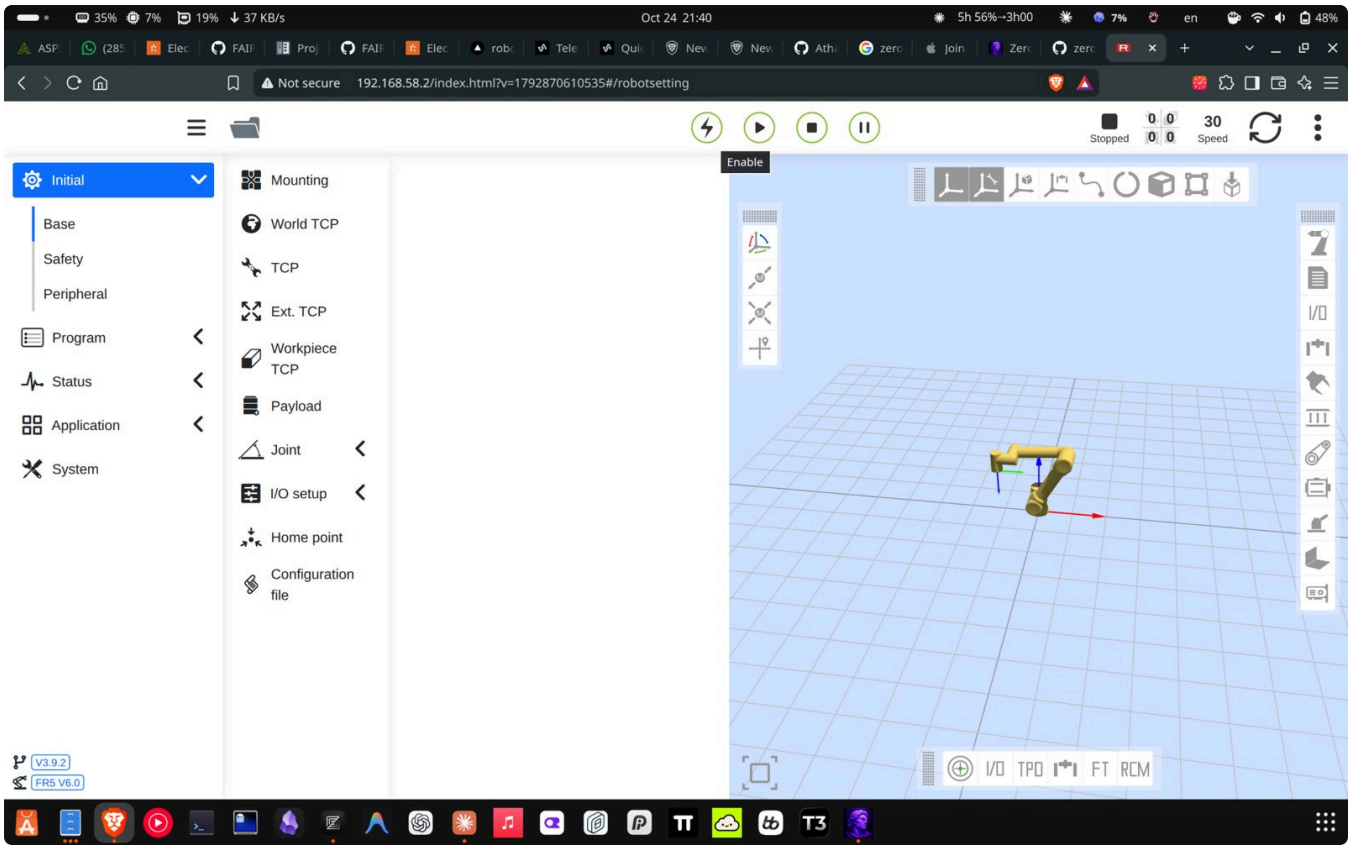
```
pixi run demo
```

Real robot

Robot prep checklist, every session:

- [ ] Ethernet cable laptop ↔ controller; `nmcli con up fr5-robot` (laptop 192.168.58.10/24); `ping 192.168.58.2` works

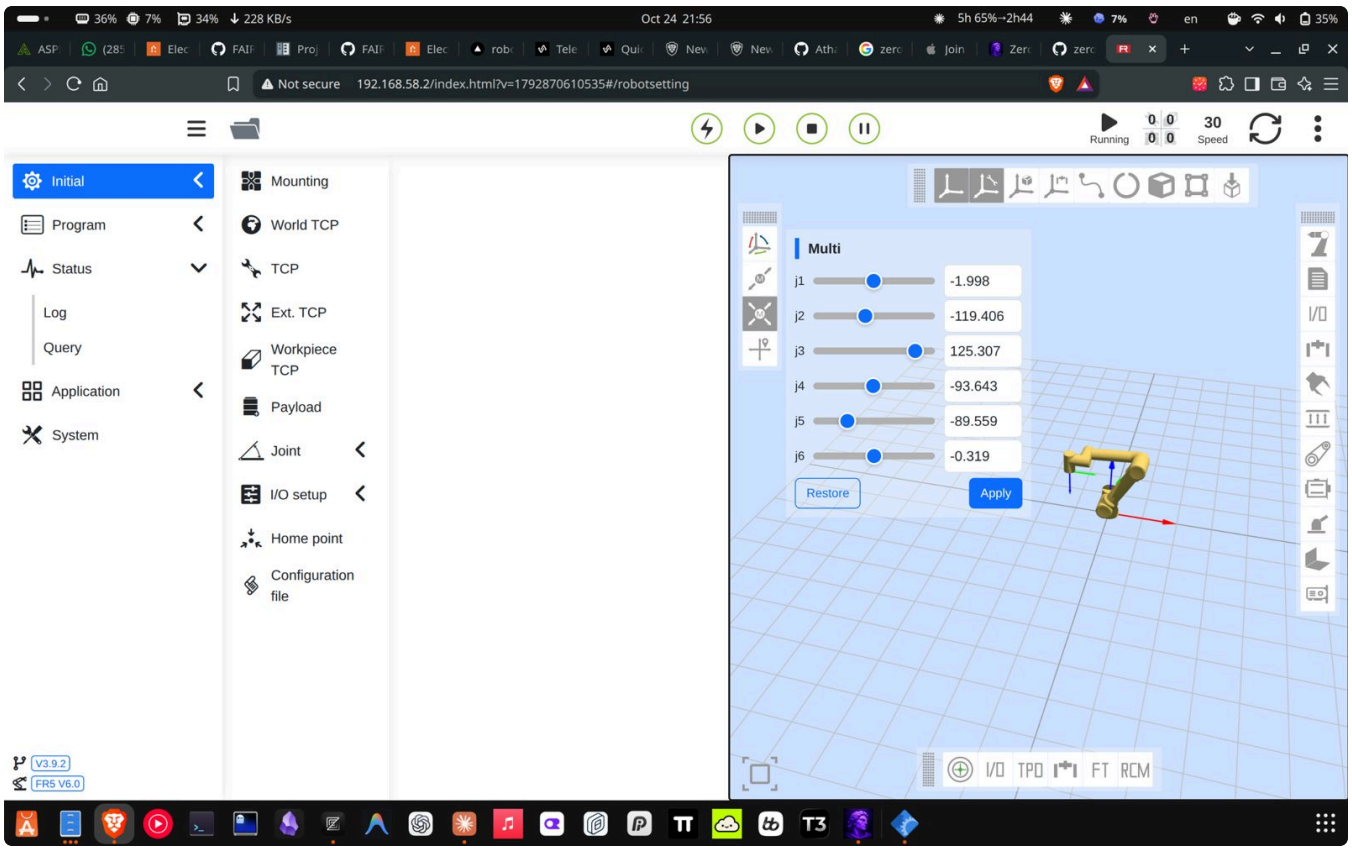
- [ ] Web UI `http://192.168.58.2`: logged in, **automatic mode** (circular-arrows icon, not the hand), robot **enabled**, no alarms
- [ ] One person at the e-stop, workspace clear
- [ ] Nobody jogging or running programs from the web UI or pendant while ROS is connected



Web UI ready for ROS: circular-arrows icon (automatic mode) at top right, Stopped, no alarm; controller V3.9.2 bottom left.

```
pixi run robot
```

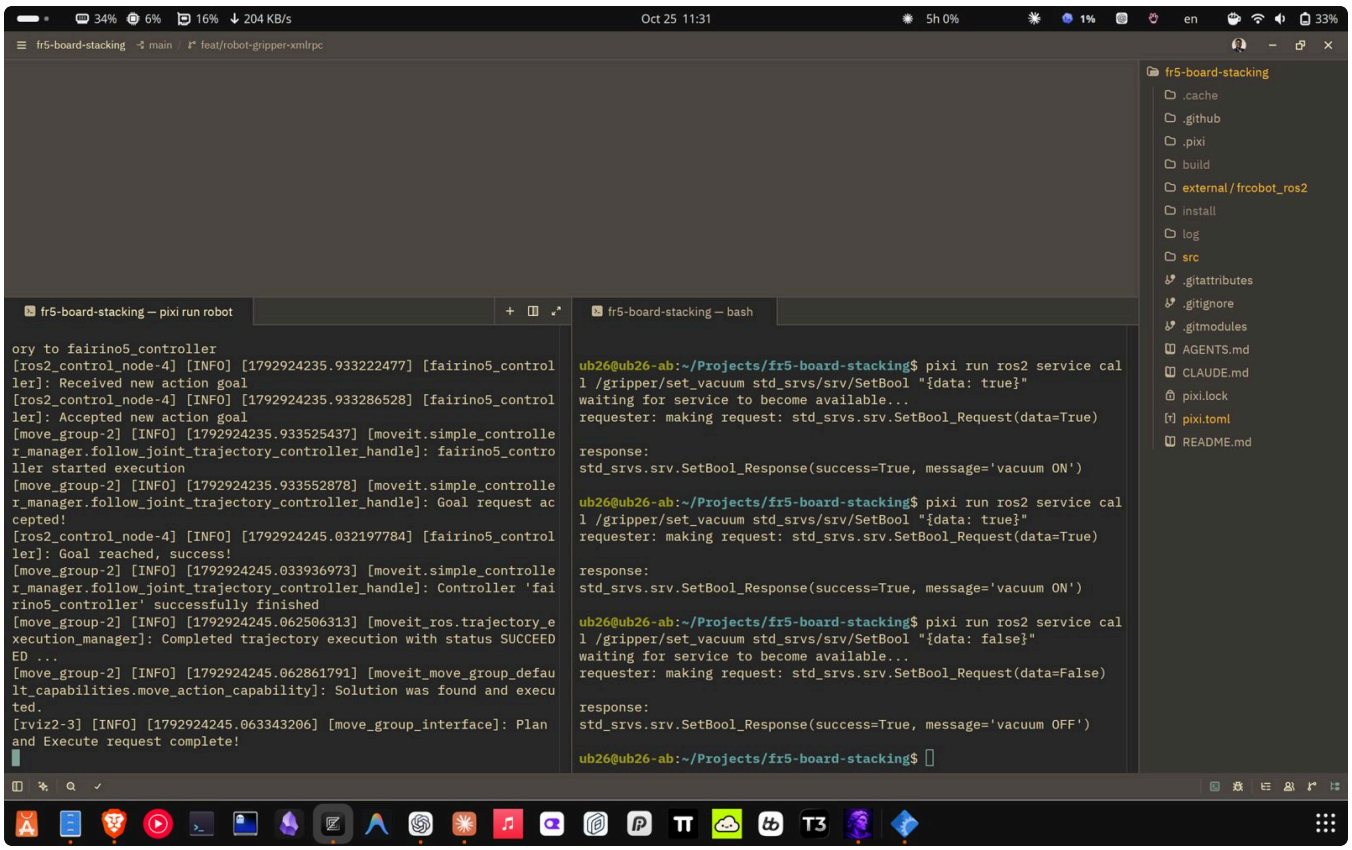
The arm holds its current pose at start. Check RViz mirrors the real arm before planning. Stop with Ctrl+C (the driver calls StopMotion and disconnects), then set manual mode and disable when leaving.



Sanity check after launch: the web UI's joint angles match `/joint_states` (radians × 57.3). Status shows Running while the driver holds the arm. Don't press Apply here while ROS is connected.

Gripper

```
pixi run ros2 service call /gripper/set_vacuum std_srvs/srv/SetBool "{data:
true}"
pixi run ros2 service call /gripper/set_vacuum std_srvs/srv/SetBool "{data:
false}"
```



Vacuum on/off from ROS on the real robot (right), with MoveIt still executing moves (left).

Repo layout

Our code lives in `src/`; Fairino's is an untouched submodule in `external/`.

Path	What it is
`external/frcobot_ros2/`	Fairino ROS 2 submodule (pinned). **Never edit.** We build 4 packages from it: description, msgs, `fairino_hardware_v3_9_0_1` (matches controller V3.9.2), FR5 MoveIt config
`src/fr5_bringup/launch/bringup.launch.py`	One launch for sim and real (`use_fake_hardware`). Starts MoveIt, ros2_control, RViz and the gripper node

Path	What it is
`src/fr5_bringup/urdf/`	Our robot description: FR5 + flange + D405 mount + gripper + `tcp`, our ros2_control block (mock vs Fairino plugin), workcell scene
`src/fr5_bringup/config/workcell.yaml`	Table, profile, post, zones, overhead camera, board size (base_link, metres)
`src/fr5_bringup/config/hardware.yaml`	Robot IP and the tool DO that switches the vacuum
`src/fr5_bringup/meshes/`	Gripper mesh (from the Schmalz STEP) and D405 mount mesh (from our STL)
`src/fr5_gripper/`	Vacuum node: `/gripper/set_vacuum` service, `/gripper/vacuum_on` topic
`pixi.toml`	Dependencies and tasks: `build`, `demo`, `robot`
`AGENTS.md`	Team and AI-agent rules (read it)

Workflow: branch per task (`feat/<area>-<desc>`), PR into `main`, squash merge. Reviews optional; AI agents never merge.

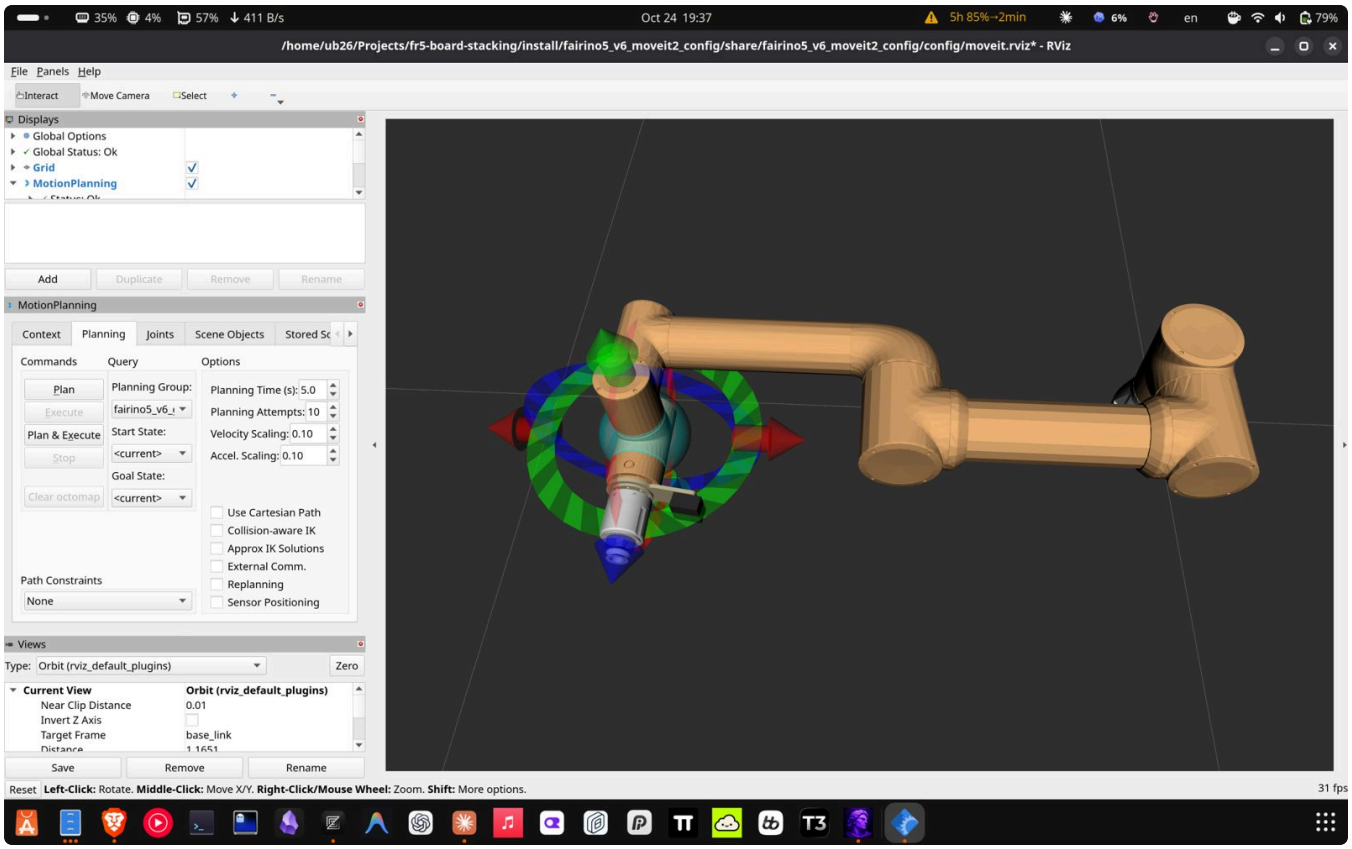
Frames and coordinates

Everything in our code and config is in **`base_link`** (metres, radians), which is rotated 180° from what the teach pendant shows.

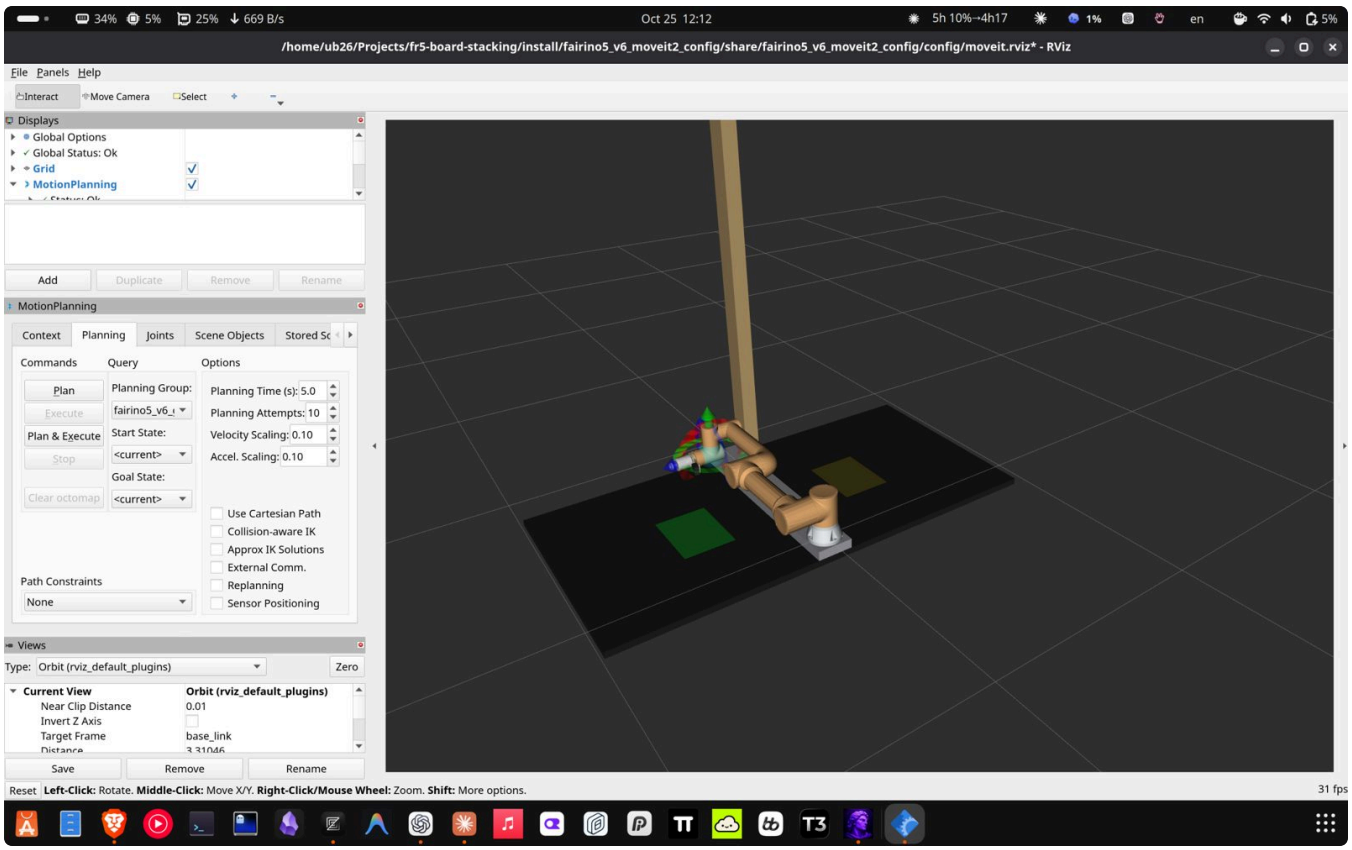
> **Pendant → base_link:** `x = −x_pendant`, `y = −y_pendant`, `z = z_pendant`; and the pendant shows **millimetres**. Verified on the robot with `tf2_echo base_link flange` against the web UI (tool 0).

Frame	Where	Notes
`base_link`	Bottom centre of the robot base, +z up	−x runs along the profile away from the near edge; −y points to the 89 cm (pick) side

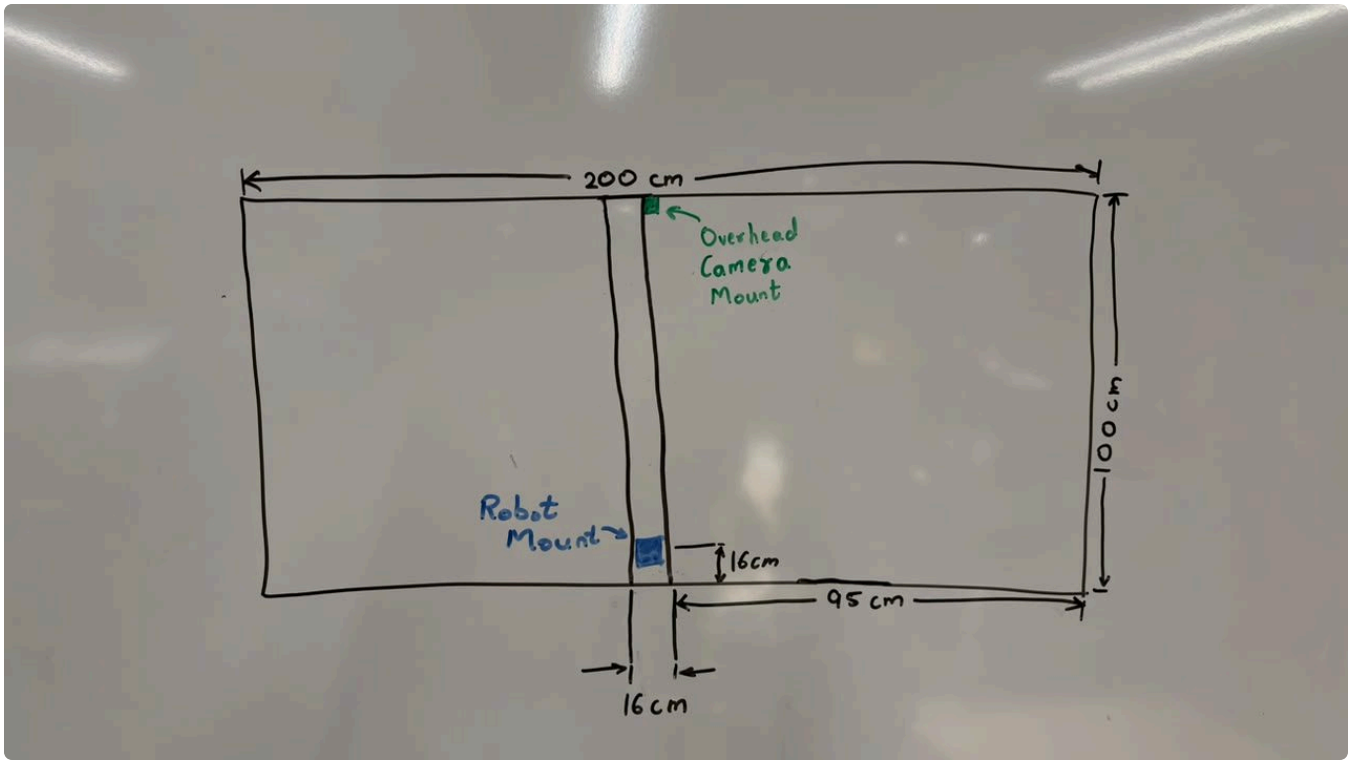
Frame	Where	Notes
`flange`	Robot flange face (wrist3_link + 100 mm)	
`tool_adapter`	Gripper mounting face, 9.5 mm above the flange	The printed D405 plate sits in between
`tcp`	Relaxed suction-cup tip, 112.9 mm from the flange, +z out of the cup	Command this frame. Pointing down: quaternion (cos(yaw/2), sin(yaw/2), 0, 0)
`wrist_camera_link`	D405 front face, x = viewing direction	Approximate; replace with hand-eye calibration
`overhead_camera_link`	Webcam, 1.61 m above the table, looking down	Approximate; replace with extrinsic calibration
`pick_zone` / `place_zone`	On the table surface	Placeholder centres (±0.40 m); set from measured positions



The tool stack in RViz: beige printed plate, dark D405 box, white gripper; `tcp` sits at the cup tip.



Workcell after the orientation fix: green = pick zone (89 cm side, −y), yellow = place zone (+y), post at the far end of the profile (−x).



The team sketch the workcell numbers come from: table 200 × 100 cm, 16 cm profile, robot mount 8 cm from the near edge, overhead camera mount at the far end.





The overhead camera post, modelled as a collision box: it stands at the edge of the robot's reach.

Decisions and gotchas

Each of these cost real debugging time; knowing them up front saves you the same.

Topic	What we do	Why
Fairino's `ros2_cmd_server`	**Not used** (`with_cmd_server` defaults to false)	It and the motion driver both need the controller's realtime port, which accepts one client. Together, the driver hangs on connect or loses its link. Its state feedback is also broken (V3.9.0 driver vs V3.9.2 controller)
Vacuum control	XML-RPC `SetToolDO(0, on, 0, 0)` on port 20003	Plain command channel, no realtime port, coexists with motion. Returns 0 on success
MoveIt from Python	`/move_action` (MoveGroup action)	Same path as RViz Plan & Execute, no extra config, fast edit-rerun loop. `moveit_py` 2.5.10 exists on RoboStack but needs its own config and launch; deferred

Topic	What we do	Why
Planners	OMPL for free moves, Pilz LIN for straight approach/retreat	OMPL is collision-aware; LIN gives predictable vertical moves
Launch file	Builds every node from **our** description	MoveIt's `generate_demo_launch()` re-reads Fairino's URDF without our gripper; that would plan without the gripper on the real robot
`libfairino.so.2` not found	`LD_LIBRARY_PATH` set in `pixi.toml` activation	The plugin needs Fairino's prebuilt library at run time
Build quirks	`-include unistd.h`, glibc 2.34 sysroot	Fairino's code relies on indirect includes; their prebuilt lib needs GLIBC 2.34
Manual mode	Always **automatic** while ROS is connected	In manual mode motion worked briefly, then the controller dropped the connection
Controller version	Driver `fairino_hardware_v3_9_0_1`	Closest driver not newer than controller V3.9.2

Safety rules for the real robot

These are not optional, and they apply to AI agents too (see `AGENTS.md`).

- **A human at the e-stop** for every real-robot run, watching the arm, not the screen.
- **Simulation first.** Every new motion script runs in `pixi run demo` before `pixi run robot`.
- **10% velocity and acceleration** for the first run of any new motion; raise only after it has worked.
- **Plan, look, then execute** for anything new: check the preview in RViz first.
- **No straight-down move onto the table until `top_z` is measured.** With a wrong table height, a LIN descent pushes the cup into the table.
- **One controller of the arm at a time:** no jogging or programs from the web UI or pendant while ROS is connected.

- **Stop options, in order:** Stop in RViz → Ctrl+C in the launch terminal → e-stop.

Your next task: P1 blind pick-and-place

Goal: one board from a **known** position into the place zone, no camera. This proves the physical cycle (tcp height, vacuum hold, place height) so vision can plug in later.

The sequence: above the board → straight down (LIN) → vacuum on → straight up → above the place zone → straight down → vacuum off → straight up. Heights come from `workcell.yaml`; the cup presses 3 mm into the board and releases 3 mm above the stack.

Steps

1. Branch `feat/task-blind-pick-place`; create `src/fr5_task/` with the four files in the appendix.
2. Add `src/fr5_task` to the end of the `--paths` list in the `build` task in `pixi.toml`; `pixi run build` (expect 7 packages).
3. **Sim:** `pixi run demo`, then:

```
pixi run ros2 run fr5_task blind_pick_place.py --x -0.40 --y -0.40
```

4. **Measure the real table height** (`top_z`): jog the relaxed cup tip until it just touches the table, read the pendant z (mm) with the tool set to our tcp offset, or read `tf2_echo base_link tcp` z. Put it in `workcell.yaml`.
5. **Real, step by step:** `--first-only` first (stops above the board), then the full run at 10%, e-stop person in place.
6. Repeat with `--stack 1`, `--stack 2` … to check stacking heights.

Done when

- [ ] Full cycle succeeds in sim (`Done.` in the log)
- [ ] On the real robot the board is lifted without slipping and released inside the boundary
- [ ] Three boards stacked with `--stack 0/1/2`
- [ ] PR merged into `main`

Note: the pick and place coordinates are `base_link` values. If you read a position off the pendant, negate x and y and convert mm to m.

Open TODOs

Ordered by how much they block a full demo.

#	Item	Where	How
1	Table height `top_z` (placeholder −0.045)	`workcell.yaml`	Touch the table with the cup tip, read z (see P1 step 4)
2	Pick and place zone centres (placeholders)	`workcell.yaml`	Jog the tcp to each centre; convert pendant → base_link
3	TCP ruler check (≈113 mm flange → relaxed cup tip)	`schmalz_gripper.xacro` `tcp_z`	Ruler; adjust if off by more than 1–2 mm
4	Camera arm yaw around the tool axis	`fr5_workcell.urdf.xacro`, `d405_wrist_mount yaw`	RViz TF display: angle from `flange` x-axis to the real arm; or hand-eye result
5	Part-present / vacuum feedback	`fr5_gripper`	Find which tool DI the Schmalz signal uses, then try reading it over XML-RPC (e.g. `GetToolDI`)
6	Attach the board in MoveIt while carried	task code	Collision object attached to `tcp` so plans account for it
7	Place guide (3D-printed corners) in the scene	`workcell_scene.xacro`	Add as boxes once printed and placed
8	Camera post size check	`workcell.yaml`	Measure; the box is generous for now

Interface with the vision work

Vision hands the task code board poses in `base_link`; the task code never touches pixels. Agreeing on this now lets both sides work in parallel.

What vision provides, per board:

Field	Meaning
`x`, `y`	Board centre in `base_link`, metres
`top_z`	Board top surface height in `base_link`, metres (measured, so stacks work)
`yaw`	Rotation about base z, radians (a rectangle is symmetric: 0–π is enough)
`layer`	Layer index, `round((top_z - table_z) / 0.006)`
`confidence`	0–1, plus a flag if the board failed the 270 × 200 mm geometry check

Integration path: P1's `blind_pick_place.py` takes `--x --y --yaw` today. Vision replaces those arguments: the task loop asks for detections, picks the highest valid board, and passes its `x, y, top_z, yaw` to the same motion code. The only change on the motion side is using the measured `top_z` instead of the table height plus one board.

Frames vision will use: `wrist_camera_link` and `overhead_camera_link` exist in TF today with approximate poses. Calibration (hand-eye for the D405, extrinsics for the webcam) replaces them.

Appendix: P1 code

Drafted and reviewed, **not yet run**: test in simulation first. Put these in `src/fr5_task/` and `chmod +x scripts/blind_pick_place.py`.

`scripts/fr5_motion.py`

```
"""Thin FR5 interface: move the tcp (cup pointing down) and switch the
vacuum.

Uses MoveIt's /move_action (the same path as RViz's Plan & Execute):
```

```
  free moves   -> OMPL (collision-aware: table, post, our robot model)
  linear moves -> Pilz LIN (straight line, for approach/retreat)
Vacuum via /gripper/set_vacuum (fr5_gripper). All poses in base_link, metres.
"""
import math

import rclpy
from geometry_msgs.msg import Pose, Quaternion
from moveit_msgs.action import MoveGroup
from moveit_msgs.msg import (Constraints, MotionPlanRequest,
MoveItErrorCodes,
                             OrientationConstraint, PlanningOptions,
PositionConstraint)
from rclpy.action import ActionClient
from shape_msgs.msg import SolidPrimitive
from std_srvs.srv import SetBool

GROUP = "fairino5_v6_group"
TCP = "tcp"
BASE = "base_link"


def tcp_down(yaw):
    """Quaternion with the tcp z-axis pointing straight down, rotated by yaw
about base z."""
    return Quaternion(x=math.cos(yaw / 2.0), y=math.sin(yaw / 2.0), z=0.0,
w=0.0)


class FR5:
    def __init__(self, node, velocity=0.1, accel=0.1):
        self.node = node
        self.velocity = velocity
        self.accel = accel
        self.move_client = ActionClient(node, MoveGroup, "move_action")
        self.vac_client = node.create_client(SetBool, "gripper/set_vacuum")
        if not self.move_client.wait_for_server(timeout_sec=10.0):
            raise RuntimeError("MoveIt /move_action not available (is bringup
running?)")
        if not self.vac_client.wait_for_service(timeout_sec=10.0):
            raise RuntimeError("/gripper/set_vacuum not available (is bringup
running?)")
```

```
    def _goal(self, x, y, z, yaw, linear, plan_only):
        target = Pose()
        target.position.x, target.position.y, target.position.z = x, y, z
        target.orientation.w = 1.0

        pos = PositionConstraint()
        pos.header.frame_id = BASE
        pos.link_name = TCP
        pos.constraint_region.primitives.append(
            SolidPrimitive(type=SolidPrimitive.SPHERE, dimensions=[0.001]))
        pos.constraint_region.primitive_poses.append(target)
        pos.weight = 1.0

        ori = OrientationConstraint()
        ori.header.frame_id = BASE
        ori.link_name = TCP
        ori.orientation = tcp_down(yaw)
        ori.absolute_x_axis_tolerance = 0.01
        ori.absolute_y_axis_tolerance = 0.01
        ori.absolute_z_axis_tolerance = 0.01
        ori.weight = 1.0

        req = MotionPlanRequest()
        req.group_name = GROUP
        req.num_planning_attempts = 5
        req.allowed_planning_time = 5.0
        req.max_velocity_scaling_factor = self.velocity
        req.max_acceleration_scaling_factor = self.accel
        req.goal_constraints.append(
            Constraints(position_constraints=[pos], orientation_constraints=
[ori]))
        if linear:
            req.pipeline_id = "pilz_industrial_motion_planner"
            req.planner_id = "LIN"
        else:
            req.pipeline_id = "ompl"

        goal = MoveGroup.Goal()
        goal.request = req
        goal.planning_options = PlanningOptions(plan_only=plan_only,
replan=False)
        goal.planning_options.planning_scene_diff.is_diff = True
        goal.planning_options.planning_scene_diff.robot_state.is_diff = True
```

```
        return goal

    def move_to(self, x, y, z, yaw=0.0, linear=False, plan_only=False,
timeout=60.0):
        kind = "LIN " if linear else "free"
        self.node.get_logger().info(
            f"move {kind} -> x={x:.3f} y={y:.3f} z={z:.3f} yaw=
{math.degrees(yaw):.1f}"
            + ("  [plan only]" if plan_only else ""))
        send = self.move_client.send_goal_async(self._goal(x, y, z, yaw,
linear, plan_only))
        rclpy.spin_until_future_complete(self.node, send, timeout_sec=10.0)
        handle = send.result()
        if handle is None or not handle.accepted:
            raise RuntimeError("MoveIt rejected the goal")
        result = handle.get_result_async()
        rclpy.spin_until_future_complete(self.node, result,
timeout_sec=timeout)
        if result.result() is None:
            handle.cancel_goal_async()
            raise RuntimeError(f"motion timed out after {timeout:.0f} s (goal
cancelled)")
        code = result.result().result.error_code.val
        if code != MoveItErrorCodes.SUCCESS:
            raise RuntimeError(f"MoveIt error code {code}")

    def vacuum(self, on):
        call = self.vac_client.call_async(SetBool.Request(data=on))
        rclpy.spin_until_future_complete(self.node, call, timeout_sec=5.0)
        res = call.result()
        if res is None or not res.success:
            raise RuntimeError(f"vacuum {'on' if on else 'off'} failed: "
                               f"{res.message if res else 'no response'}")
```

`scripts/blind_pick_place.py`

```
#!/usr/bin/env python3
"""Blind pick-and-place: one board from a known position into the place zone.

No camera. Proves the physical cycle: tcp height, vacuum hold, place height.
Heights come from fr5_bringup/config/workcell.yaml.
```

```
  ros2 run fr5_task blind_pick_place.py --x -0.40 --y -0.40 [--yaw 0] [--
stack 0]
  --stack N       boards already stacked in the place zone (place height)
  --first-only    plan AND execute only the first move (above the board),
then stop
  --velocity V    velocity/acceleration scaling (default 0.1)
"""
import argparse
import math
import os
import sys
import time

import rclpy
import yaml
from ament_index_python.packages import get_package_share_directory

from fr5_motion import FR5

APPROACH = 0.10      # approach/retreat height above the target, m
PRESS = 0.003        # push the cup this far below the board top (bellows
compress), m
RELEASE_GAP = 0.003  # release this far above the stack top, m
SETTLE = 0.5         # wait after switching the vacuum, s


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--x", type=float, required=True)
    ap.add_argument("--y", type=float, required=True)
    ap.add_argument("--yaw", type=float, default=0.0, help="board yaw,
degrees")
    ap.add_argument("--stack", type=int, default=0)
    ap.add_argument("--first-only", action="store_true")
    ap.add_argument("--velocity", type=float, default=0.1)
    args = ap.parse_args(rclpy.utilities.remove_ros_args(args=sys.argv)[1:])

    cfg_path = os.path.join(get_package_share_directory("fr5_bringup"),
"config", "workcell.yaml")
    with open(cfg_path) as f:
        cfg = yaml.safe_load(f)
    table_z = cfg["table"]["top_z"]
    thick = cfg["board"]["size"][2]
```

```
    place_x, place_y = cfg["place_zone"]["center_xy"]
    place_yaw = cfg["place_zone"]["yaw"]

    pick_z = table_z + thick - PRESS
    place_z = table_z + (args.stack + 1) * thick + RELEASE_GAP
    yaw = math.radians(args.yaw)

    rclpy.init()
    node = rclpy.create_node("blind_pick_place")
    log = node.get_logger()
    try:
        robot = FR5(node, velocity=args.velocity, accel=args.velocity)
        log.info(f"pick at ({args.x:.3f}, {args.y:.3f}) z={pick_z:.3f}; "
                 f"place at ({place_x:.3f}, {place_y:.3f}) z={place_z:.3f}")

        robot.move_to(args.x, args.y, pick_z + APPROACH, yaw)
        if args.first_only:
            log.info("--first-only: stopping above the board.")
            return
        robot.move_to(args.x, args.y, pick_z, yaw, linear=True)
        robot.vacuum(True)
        time.sleep(SETTLE)
        robot.move_to(args.x, args.y, pick_z + APPROACH, yaw, linear=True)

        robot.move_to(place_x, place_y, place_z + APPROACH, place_yaw)
        robot.move_to(place_x, place_y, place_z, place_yaw, linear=True)
        robot.vacuum(False)
        time.sleep(SETTLE)
        robot.move_to(place_x, place_y, place_z + APPROACH, place_yaw,
linear=True)
        log.info("Done.")
    except Exception as exc:
        log.error(f"ABORTED: {exc}")
    finally:
        node.destroy_node()
        rclpy.try_shutdown()


if __name__ == "__main__":
    main()
```

package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>fr5_task</name>
  <version>0.1.0</version>
  <description>Pick-and-place task code for the FR5 board stacking demo.
</description>
  <maintainer email="athborpe@gmail.com">AtharvaBhorpe</maintainer>
  <license>TODO</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>moveit_msgs</exec_depend>
  <exec_depend>shape_msgs</exec_depend>
  <exec_depend>std_srvs</exec_depend>
  <exec_depend>fr5_bringup</exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(fr5_task)

find_package(ament_cmake REQUIRED)

install(PROGRAMS scripts/blind_pick_place.py DESTINATION lib/${PROJECT_NAME})
install(FILES scripts/fr5_motion.py DESTINATION lib/${PROJECT_NAME})

ament_package()
```